

21世纪软件工程专业规划教材

软件工程导论（第6版）

第3章 需求分析

- 为了开发出真正满足用户需求的软件产品，首先必须知道**用户的需求**。对软件需求的**深入理解**是软件开发工作获得成功的前提条件，不论人们把设计和编码工作做得如何出色，不能真正满足用户需求的程序只会令用户失望，给开发者带来烦恼。
- **需求分析**是软件定义时期的最后一个阶段，它的基本任务是准确地回答“**系统必须做什么**”这个问题，需求分析阶段最终产出是“**软件需求规格说明书**”，以书面形式准确描述软件需求。

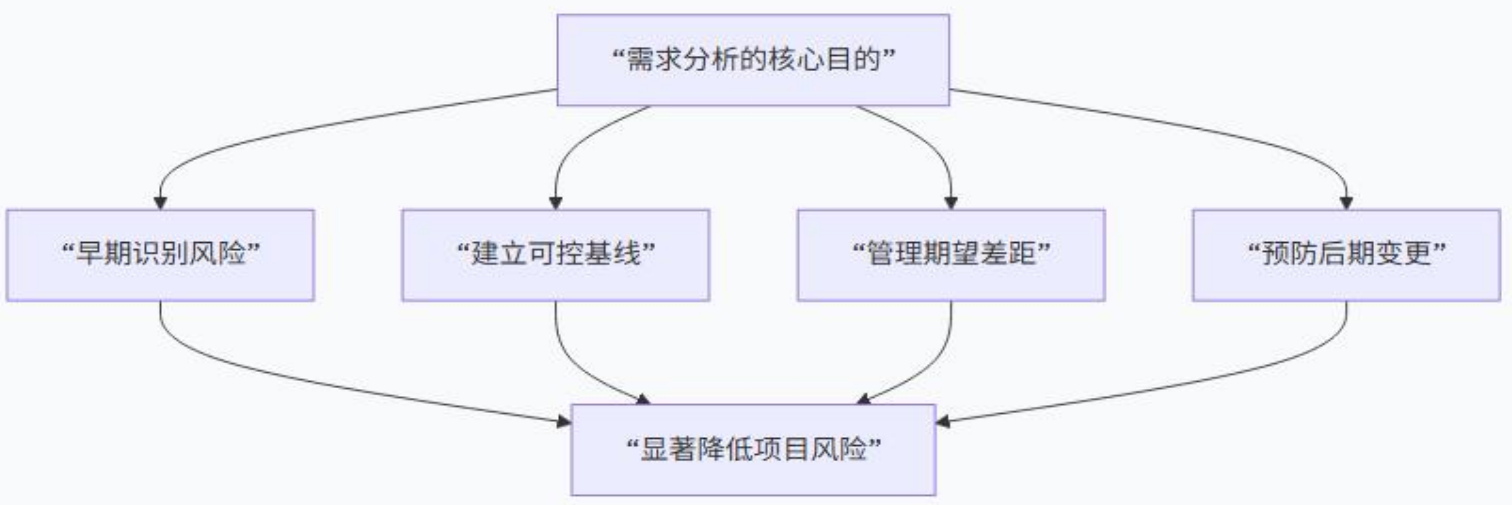
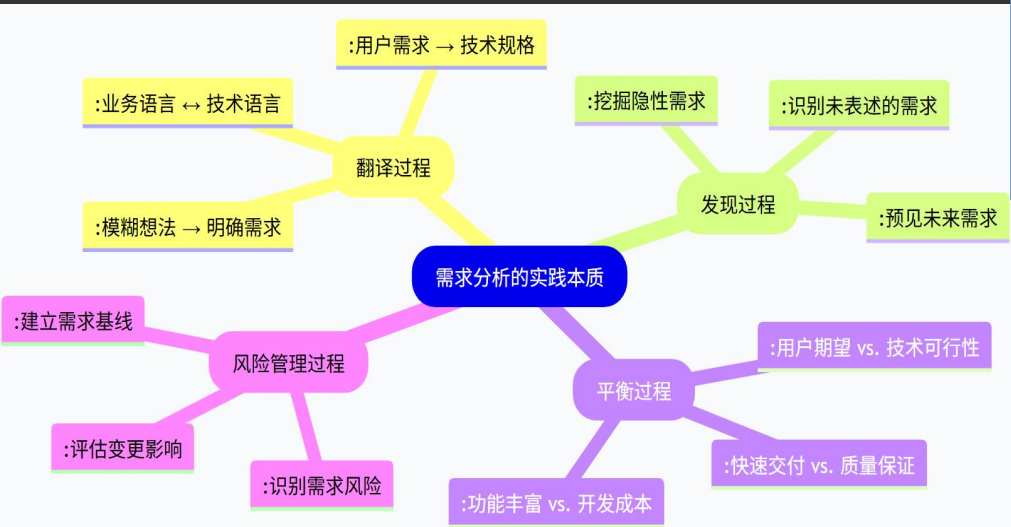
目录

CONTENTS

- 1 需求分析的任务
- 2 需求获取的方法
- 3 实体-联系图
- 4 状态转换图
- 5 软件需求验证

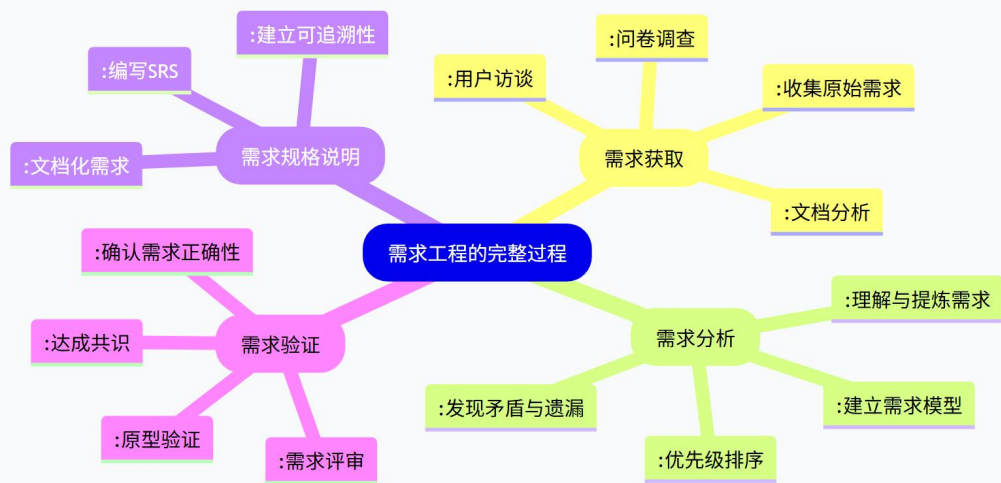
需求分析

需求分析是系统工程和软件工程中的一个正式过程。是系统化地研究、提炼、文档化和验证用户对软件系统的**需要和约束**的过程。为系统开发建立一致、完整、准确且可行的需求集合。它回答一个根本问题：“**系统到底应该做什么？**”



表达性	显性需求	用户能明确说出	"需要登录功能"	稳定性	核心需求	长期不变，业务本质	"电商系统需要购物车"
	隐性需求	用户未意识但必要	"密码需要加密存储"		可变需求	随业务变化	"促销活动的具体规则"
	兴奋型需求	超出用户预期	"登录后个性化问候"		临时需求	短期特定需求	"双11特价活动功能"
抽象层次	业务需求	战略层面	"提高市场份额5%"				
	用户需求	使用层面	"快速找到商品"				
	系统需求	实现层面	"搜索响应时间<2秒"				

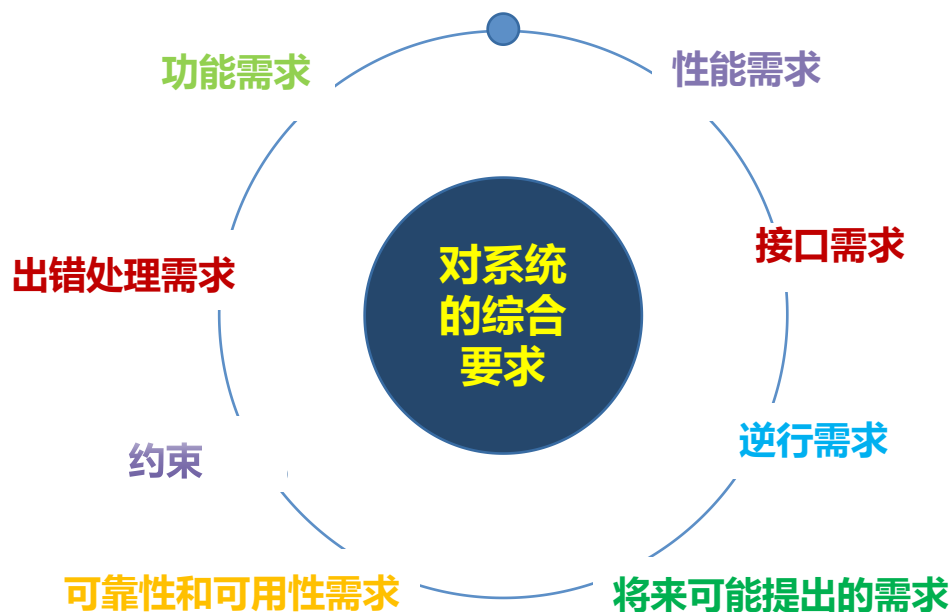
需求分析的任务



尽管目前有许多不同的用于需求分析的结构化分析方法，但是，所有这些分析方法都**遵守下述准则**：

- ① 理解并描述问题的信息域，建立**数据模型**。
- ② 定义软件应完成的功能，建立**功能模型**。
- ③ 描述作为外部事件结果的软件行为，建立**行为模型**。
- ④ 对描述信息、功能和行为的模型进行分解，**用层次的方式展示细节**。

① 确定对系统的综合要求



- ② **分析系统的数据要求**：任何一个软件系统本质上都是**信息处理系统**，系统必须处理的信息和系统应该产生的信息在很大程度上决定了系统的面貌，对软件设计有深远影响，因此，必须分析系统的数据要求，这是软件需求分析的一个重要任务。常用工具有**层次方框图**和**Warnier图**等。
- ③ **导出系统逻辑模型**：综合上述两项分析的结果导出系统的详细的逻辑模型，通常用**数据流图**、**实体-联系图**、**状态转换图**、**数据字典**和主要的**处理算法**来描述。

需求获取的方法

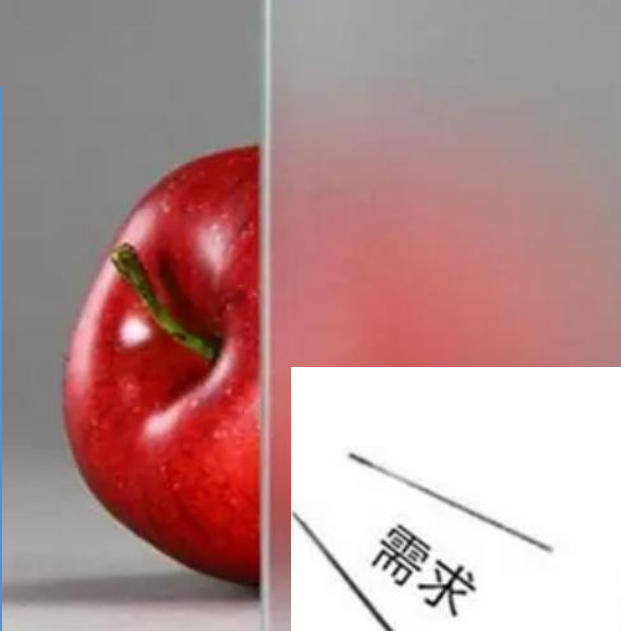
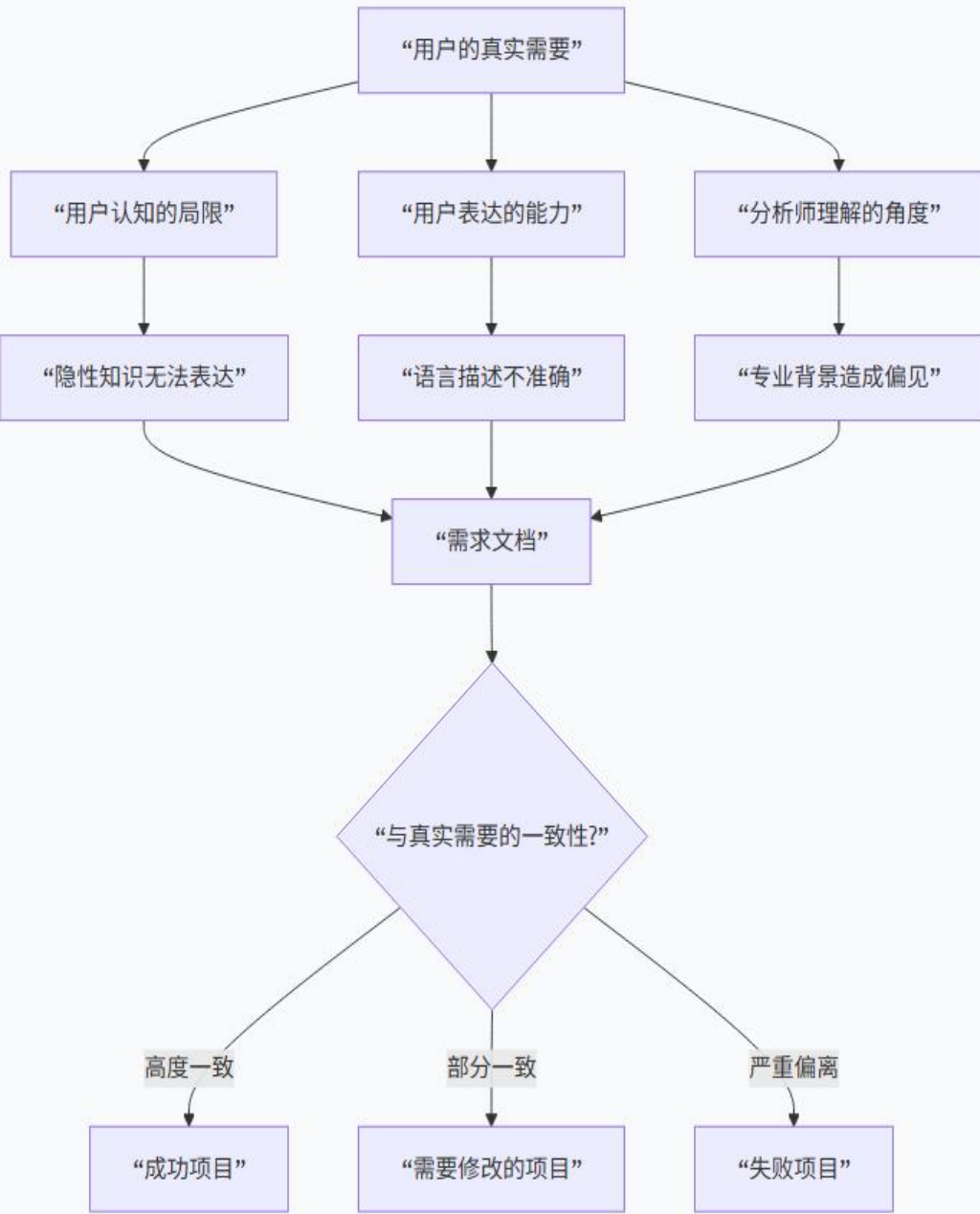
需求获取是指通过与用户的**交流**，对现有系统的观察及对任务的分析，从而开发、捕获和修订用户的需求。

需求获取需要和**用户（领导层、业务人员）**进行大量的沟通，但**沟通的重点**各自不同。和**领导**沟通主要是了解大体的业务流程、组织架构及业务需求；和**业务人员**则主要了解详细的业务流程、既有的软硬件环境、现有的运行系统、功能性需求和非功能性需求。通常应该与这些业务人员保持畅通的沟通渠道。

技术	适用场景	技巧
用户访谈	深度理解核心用户需求	准备问题清单，保持开放，追问“为什么”
问卷调查	大规模收集用户意见	问题明确，选项完整，控制长度（<10分钟）
文档分析	理解现有系统/业务流程	分析现有文档、表格、报告、邮件
现场观察	理解实际工作流程	不干扰，做笔记，事后访谈澄清
原型演示	激发用户表达需求	低保真原型，强调“这不是最终产品”
头脑风暴	创新功能想法	不评判，鼓励发散，后期收敛
用例研讨会	识别系统边界和参与者	邀请所有关键利益相关者

形成访谈记录、调查结果、观察报告、原始需求列表

用户访谈



模糊性



变化性

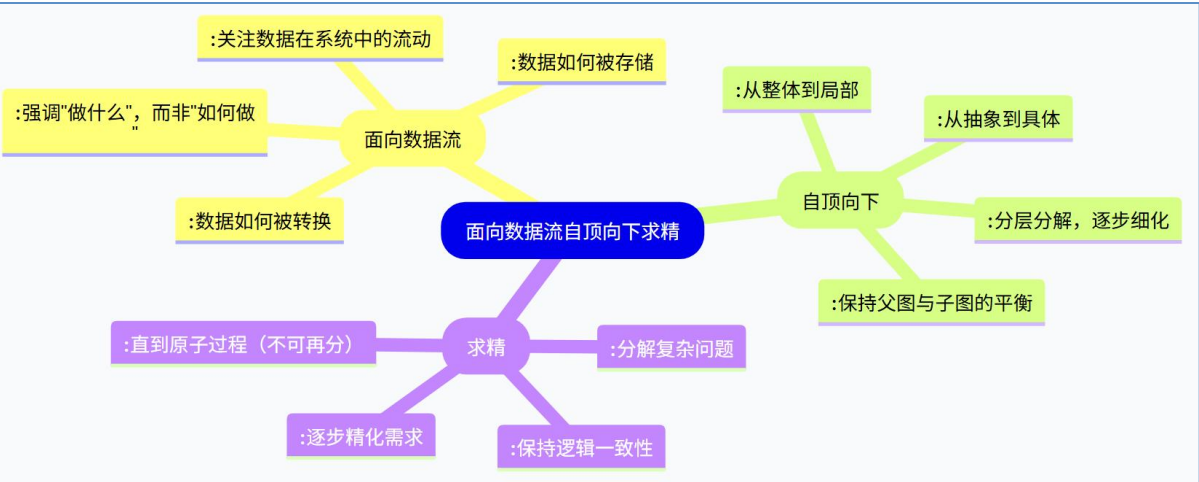


硬糖社区
BBS.WISEJOY.COM



面向数据流自顶向下求精

结构化分析方法就是数据流自顶向下求精进行需求分析的方法，它将复杂系统分解为可管理的层次，专注于数据流动而非控制流。



平衡性

①

②

③

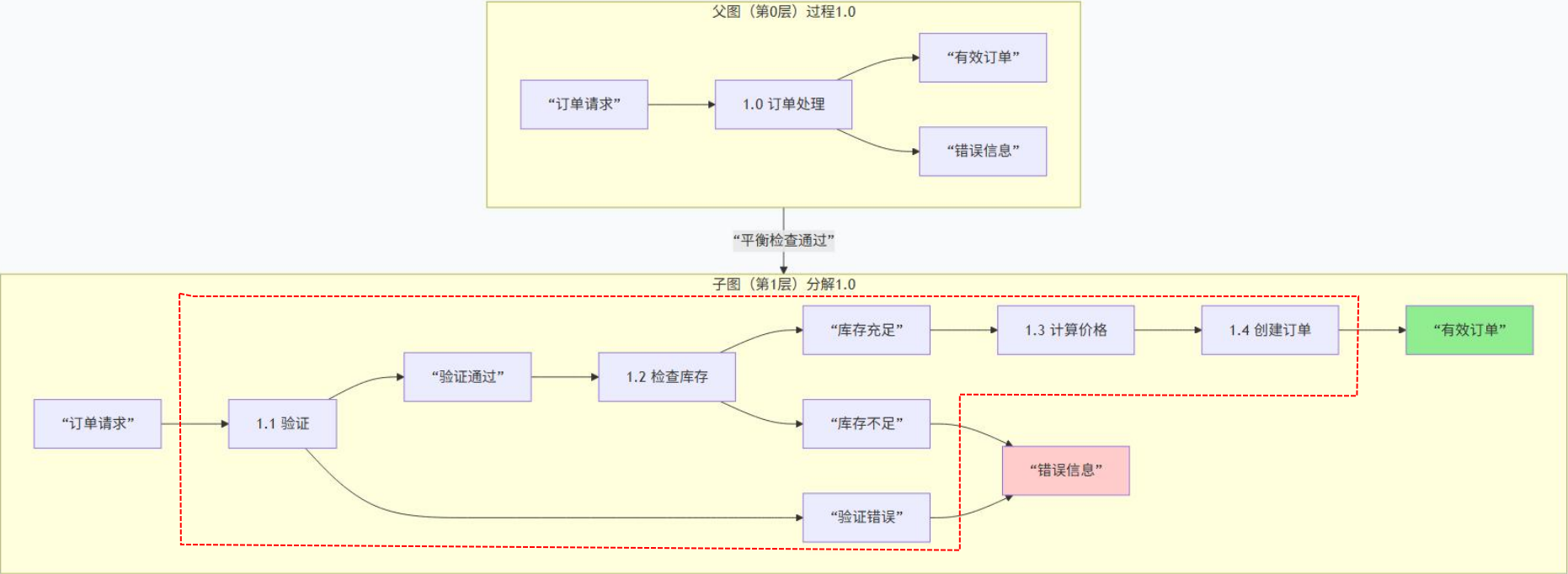
④

“1. 建立系统上下文图
(确定系统边界)”

“2. 创建第0层DFD
(识别主要过程与数据流)”

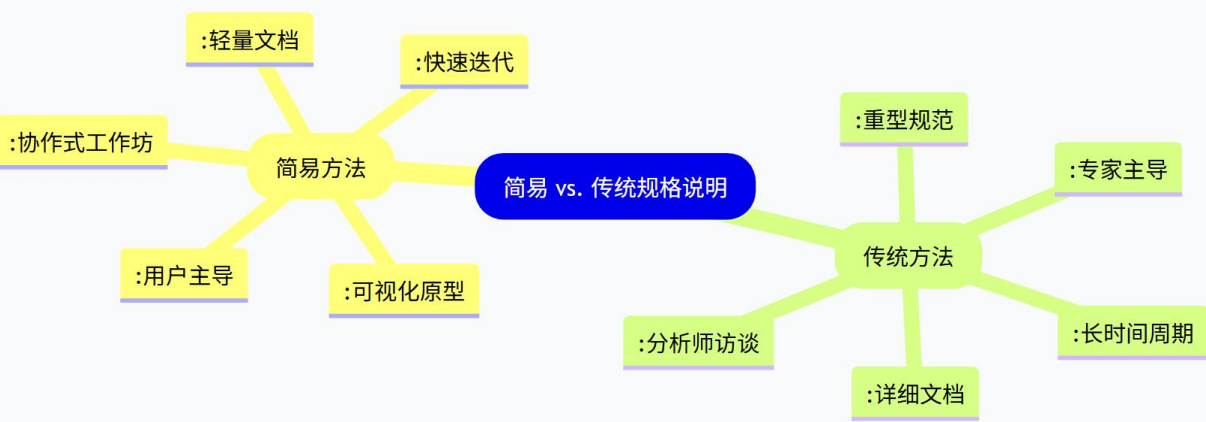
“3. 分解主要过程
(第1层及以下DFD)”

“平衡性检查
(确保父图与子图一致)”



简易的应用规格说明技术

简易的应用规格技术是一种**面向团队**的需求方法，是一种轻量级、用户友好的需求收集和规格说明方法。它强调**用户参与**和**可视化协作**，旨在快速达成对系统功能的共同理解。



- 1

• 进行初步访谈
- 2

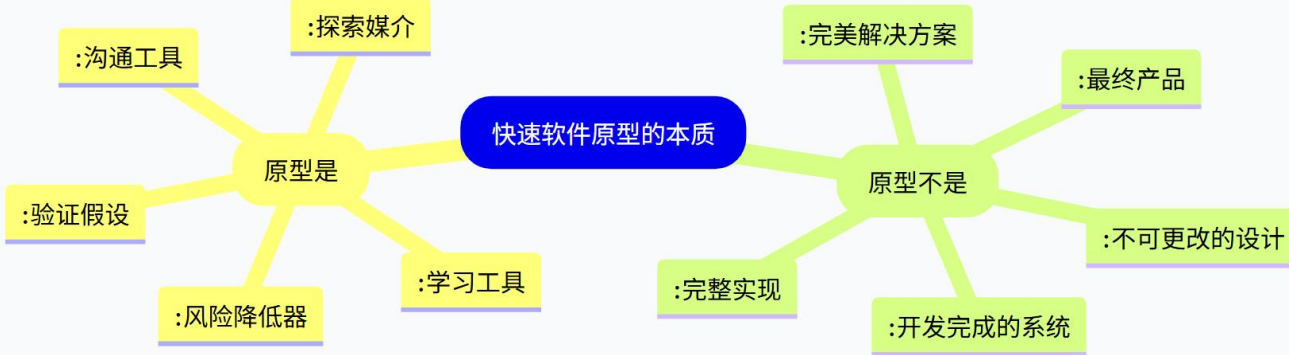
• 开发者和用户分别写出“产品需求”
- 3

• 开会讨论，各自展示需求列表
- 4

• 得出了意见一致，为需求列表制定小型规格说明
- 5

• 根据会议结果，起草完整的软件需求规格说明

快速建立软件原型



类型	制作工具	制作时间	适合阶段	验证内容
纸面原型	纸、笔、便利贴	30分钟-2小时	概念探索	流程、布局、信息架构
数字线框图	Balsamiq、Whimsical	2-8小时	需求细化	功能布局、用户流程
交互原型	Figma、Adobe XD	8-40小时	设计验证	交互体验、用户路径
高保真视觉稿	Sketch、Figma	16-80小时	视觉设计	视觉风格、品牌传达
可运行原型	代码框架（如React）	1-5天	技术验证	技术可行性、性能

原型的四个关键特性：

- 快速：几小时到几天
- 廉价：低成本制作，低成本修改
- 聚焦：只验证关键假设，不求完整
- 可抛弃：心态上准备好丢弃和重做

验证想法可行性 → 纸面原型

确定信息架构 → 数字线框图

测试用户体验 → 交互原型

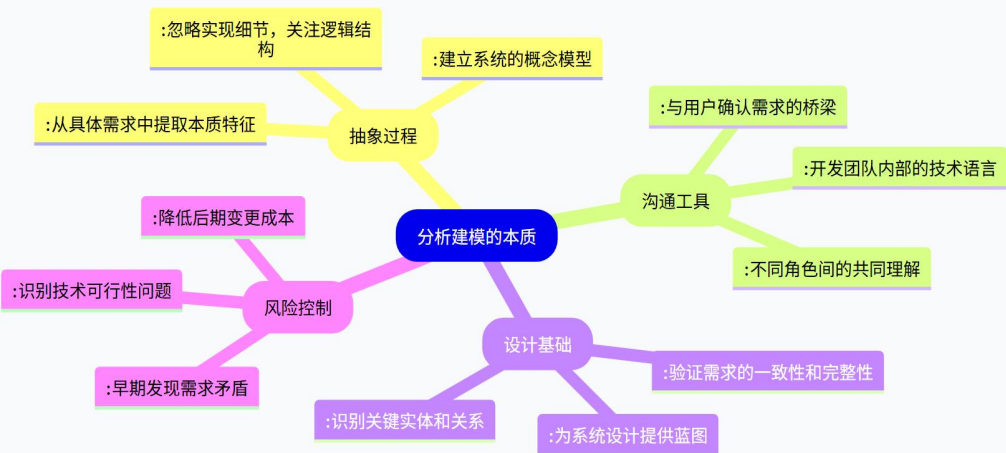
验证视觉效果 → 高保真视觉稿

验证技术方案 → 可运行原型

分析建模与规格说明

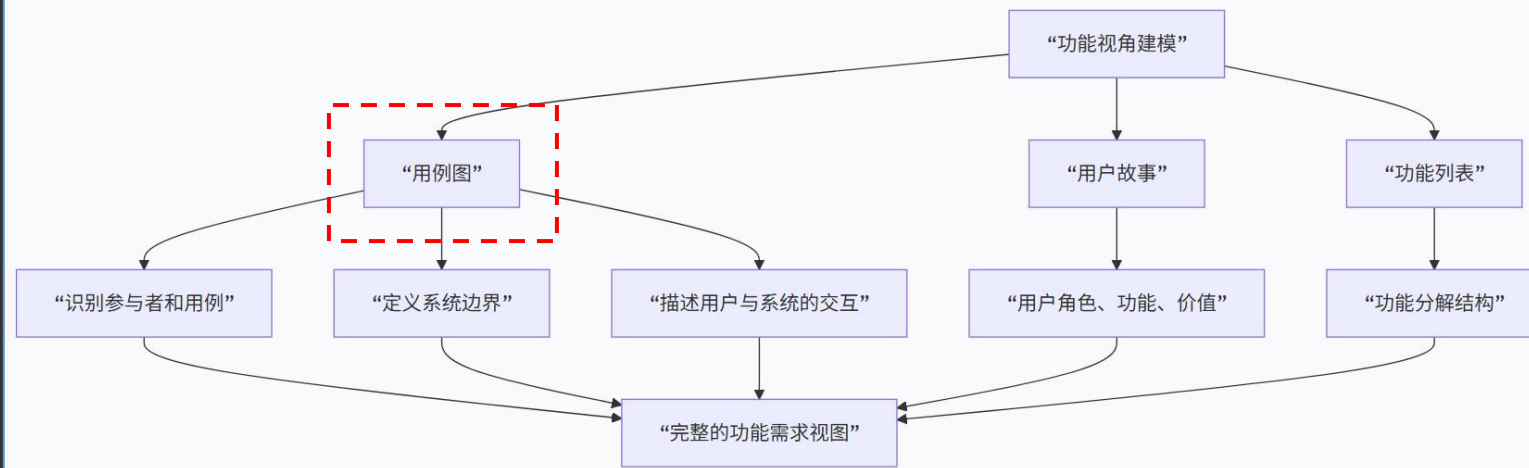
结构化分析实质上是一种**创建模型**的活动，分析建模是在需求分析阶段，使用图形化、形式化的模型来表示系统的逻辑结构和行为的过程。它是需求的理解、精化和表达。

需求分析过程应该建立三种模型：**数据模型**、**功能模型**和**行为模型**。

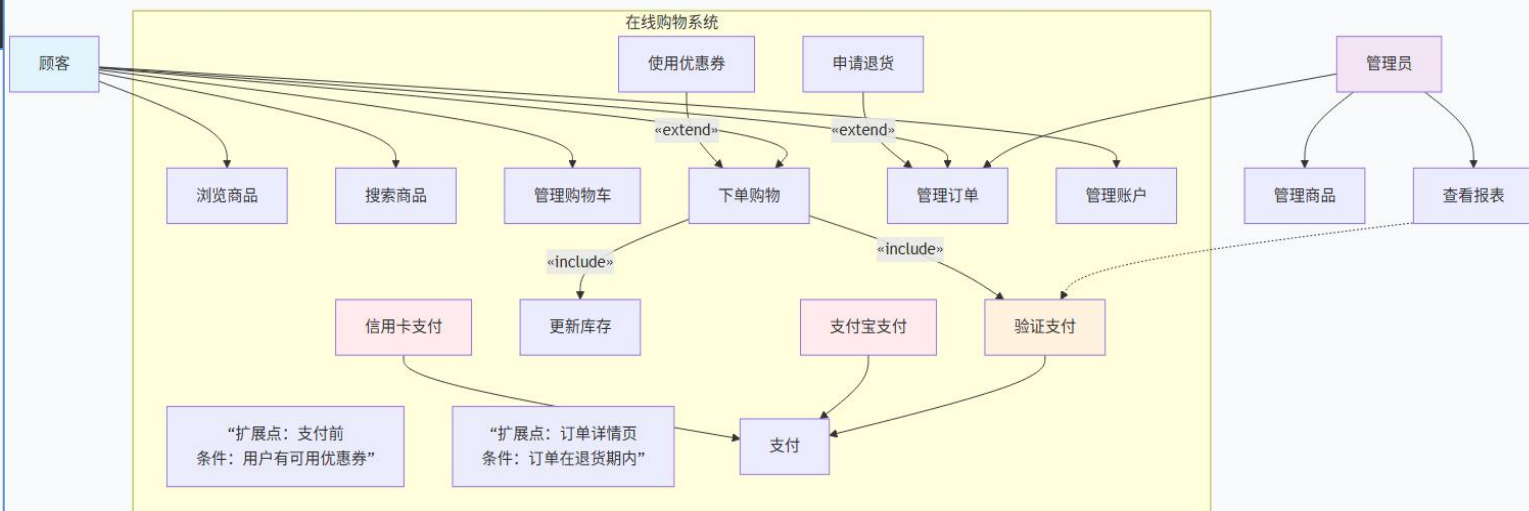


完整的分析建模需要从三个互补的视角来审视系统

功能视角：系统能做什么？——用例图/数据流图——建立功能模型

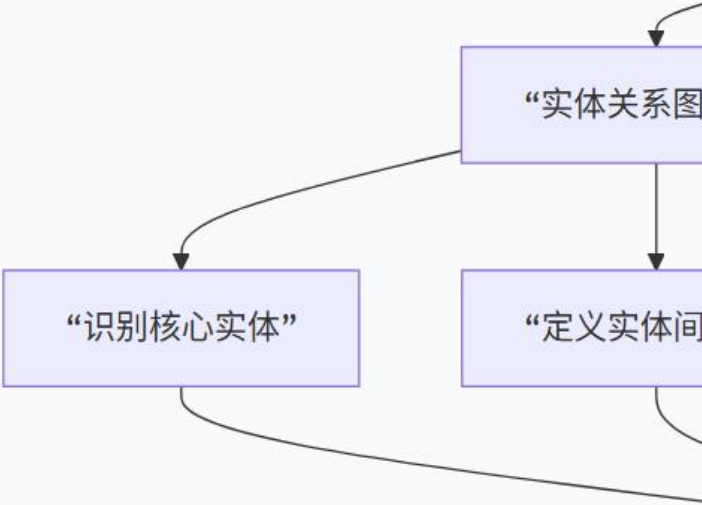


谁使用系统？（参与者） 系统做什么？（用例） 系统和外部世界如何交互？（关系）



数据流图指明了系统具有的变换数据的功能，因此**数据流图**是建立功能模型的基础

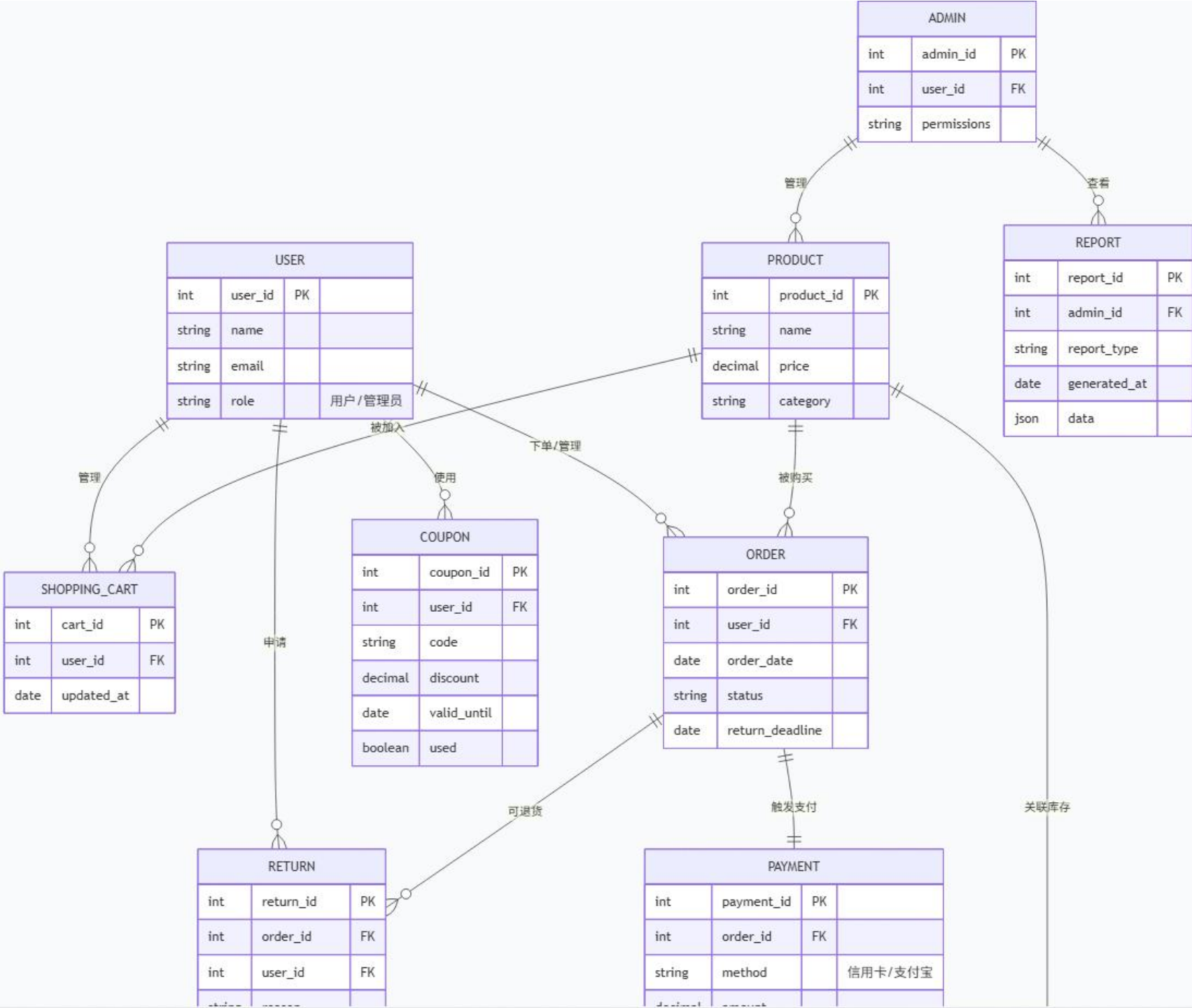
数据视角：系统处理什么信息？—— 实体关系图



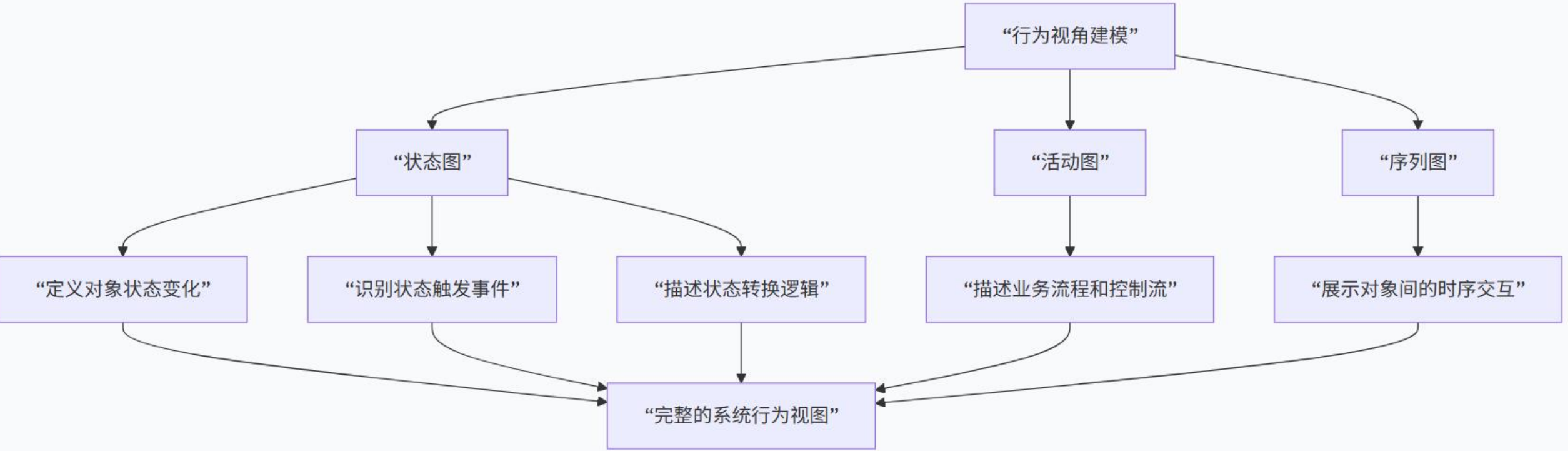
关键工具：实体关系图（ERD）：展示数据实

数据字典：定义所有数据元素的详细信息

类图（分析级）：面向对象系统中的领域模型



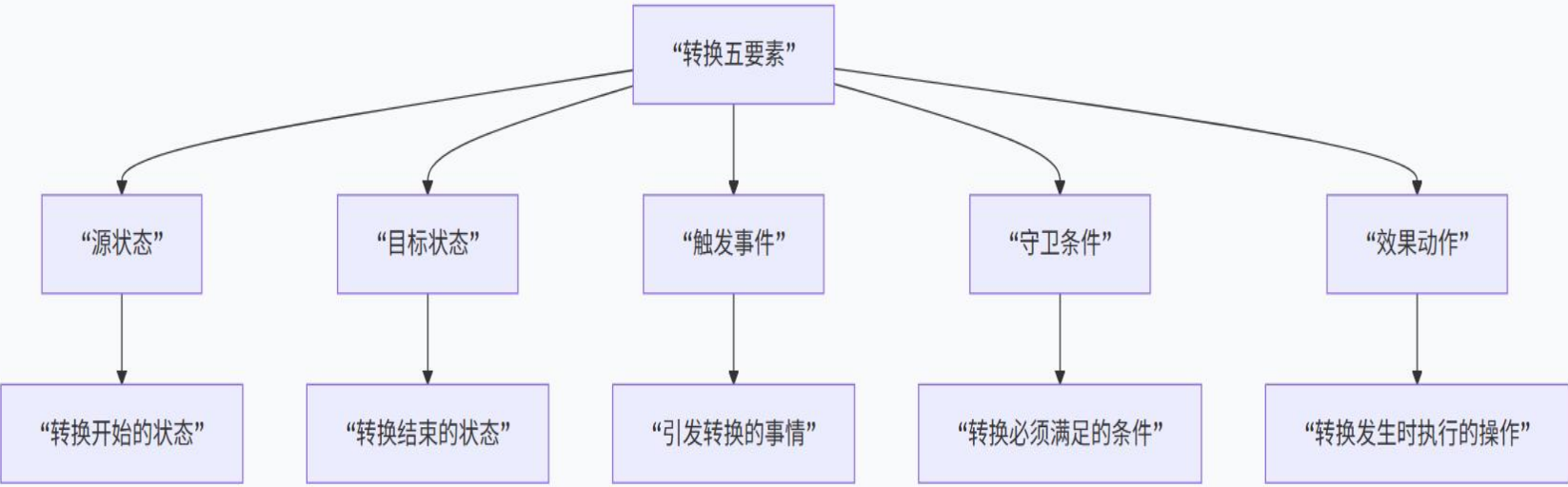
行为视角：系统如何响应事件？—— 状态转换图——行为模型



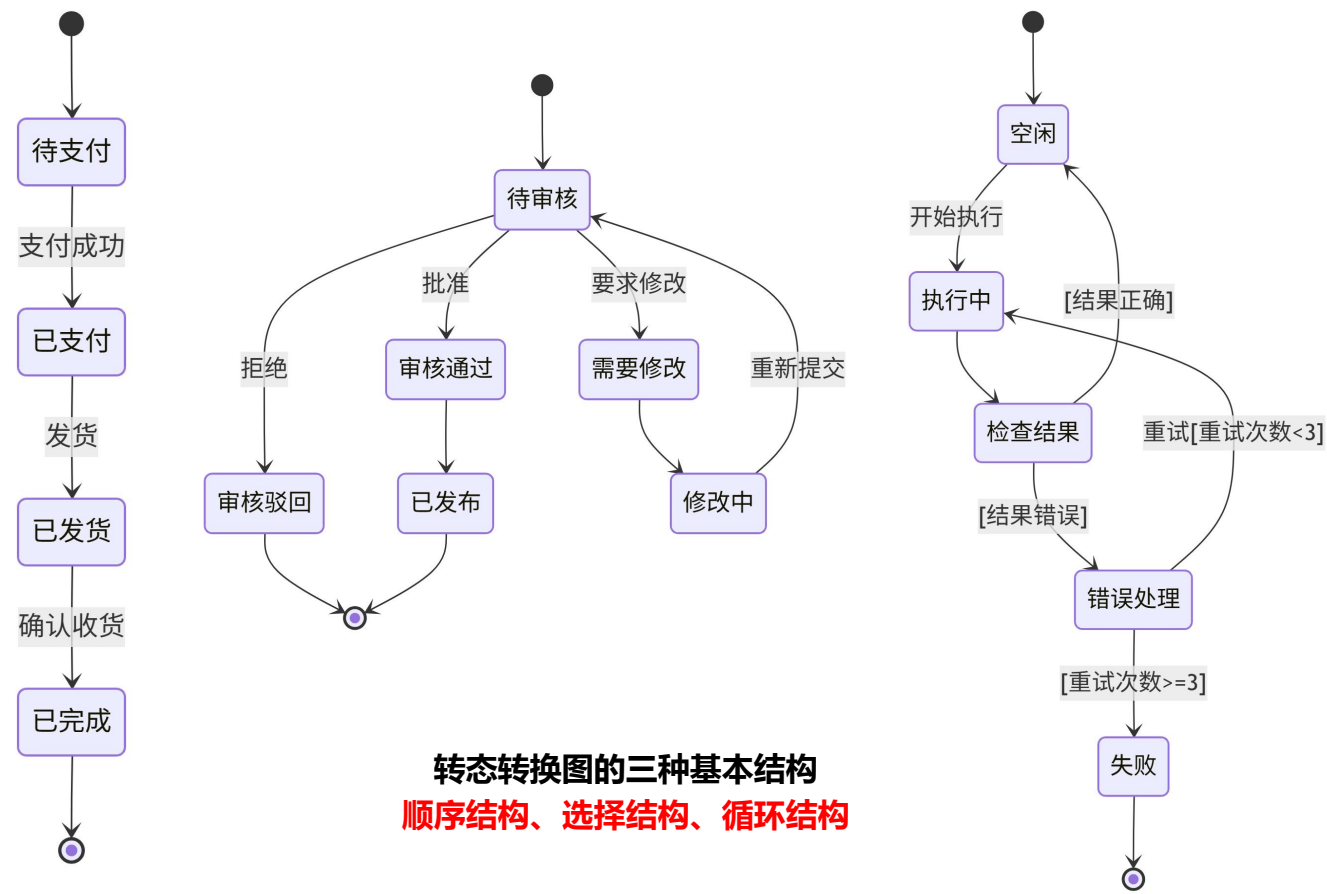
状态：是任何可以被观察到的**系统行为模式**，一个状态代表系统的一种行为模式。状态规定了系统对事件的响应方式，状态主要有：**初态**（即初始状态）、**终态**（即最终状态）和**中间状态**。在一张状态图中只能有一个初态，而终态则可以有0至多个。

事件：是在某个特定时刻发生的事情，它引起系统做动作或（和）转换状态的**控制信息**。

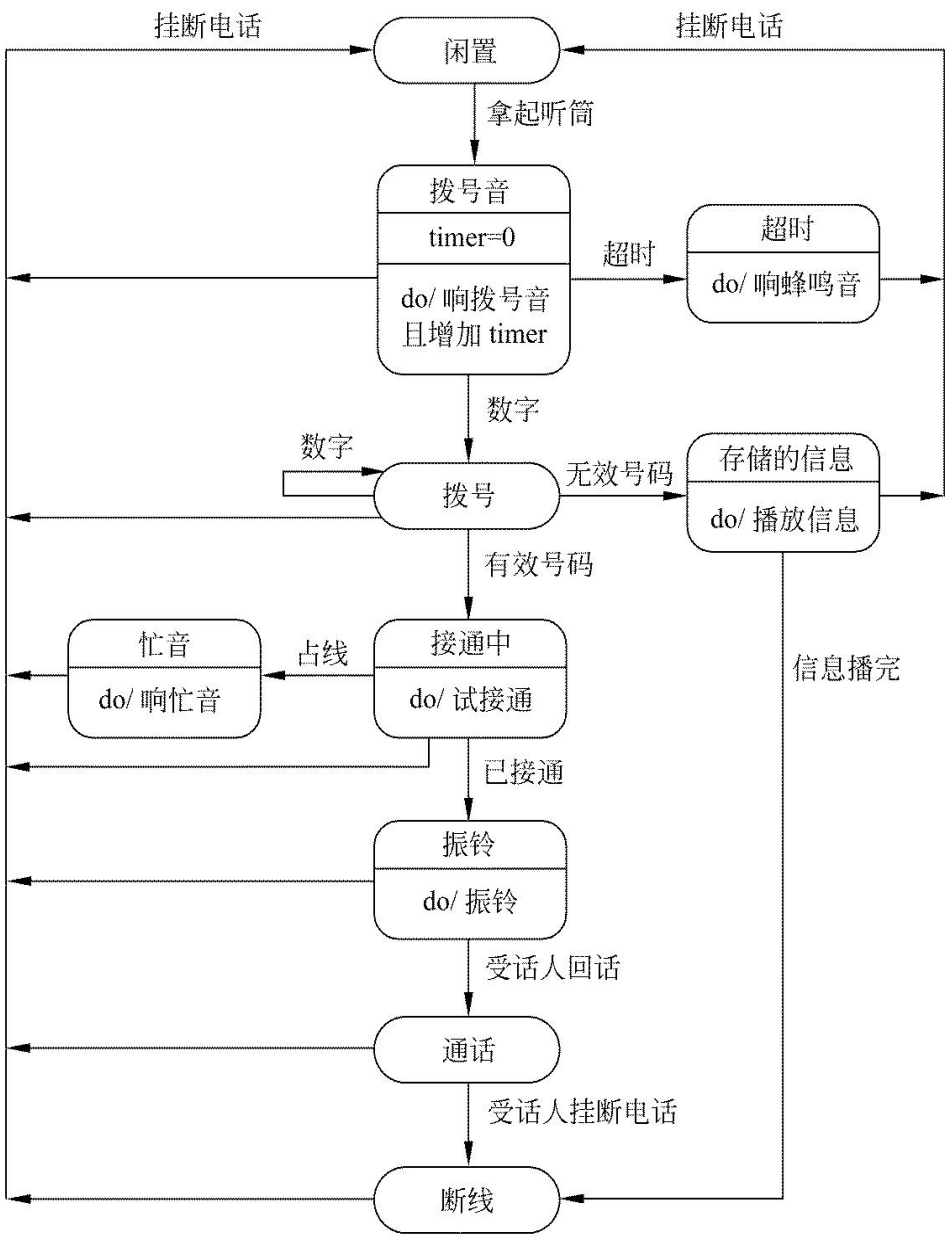
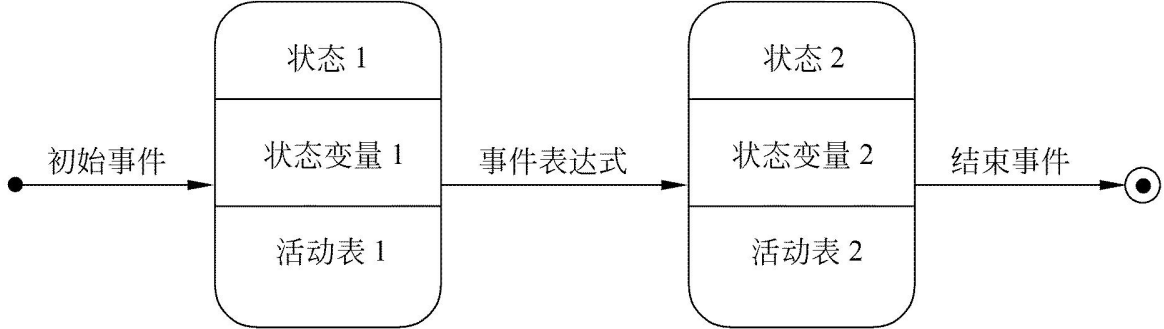
转换：是状态之间的有向关系，表示当特定事件发生且满足特定条件时，对象从源状态进入目标状态。



行为视角：系统如何响应时间？—— 状态转换图——行为模型



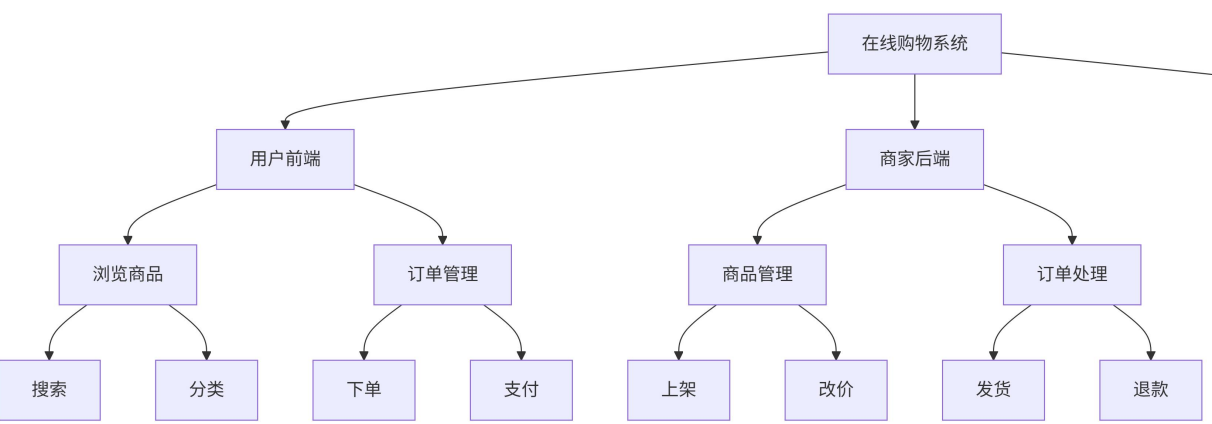
转态转换图的三种基本结构
顺序结构、选择结构、循环结构



电话系统的状态图

层次方框图

层次方框图是一种用于描述系统、数据或概念的层次结构的树状图表。它以逐层分解、自顶向下的方式，清晰地展现整体与部分的关系。



- ① **清晰的树状层次结构**：图形像一棵倒置的树，顶层是根节点（总系统/总概念），向下分支为各层子节点（子系统/组成部分）。连线表示“包含”、“组成”或“分解为”的关系，而非控制流或数据流。
- ② **自顶向下的分解方法**：从最抽象、最概括的顶层（总览）开始，逐层细化，直到达到所需的详细程度（如基本功能、不可再分的数据项）。
- ③ **模块化与封装**：每个方框及其下层结构都可以看作一个相对独立的模块。便于理解局部细节而不必立刻面对全局复杂性。
- ④ **强调静态结构，而非动态过程**：主要回答“系统/数据由什么组成？”，而不是“系统如何运行？”。因此，它与描述流程的流程图和描述状态变化的状态图有本质区别。

Warnier图

Warnier图由法国计算机科学家提出的一种系统设计与程序结构描述工具。它采用**花括号**来清晰地表示数据的层次结构和程序的逻辑构造，特别强调**输入-处理-输出**的关系。

```
销售订单 {
  订单头 {
    订单号
    客户信息
    订单日期
  }
  订单明细 {
    (1, n) 订单行 { // 循环，至少1行，最多n行
      商品编码
      数量
      单价
      (0,1) 折扣 // 可选
    }
  }
  订单尾 {
    商品总金额
    (0,1) 运费
    订单总额
  }
}
```

```
text

处理订单 {
  读入订单头
  初始化总金额
  处理订单明细 {
    (n) { // 循环n次，对应n个订单行
      读入一行明细
      计算行合计 (数量*单价)
      ⊕ 有折扣：应用折扣
      ⊕ 无折扣：不变
      累加到总金额
    }
  }
  计算运费 {
    ⊕ [总金额 > 100]：运费 = 0
    ⊕ [总金额 <= 100]：运费 = 10
  }
  写订单总额 (总金额+运费)
}
```

特性	Warnier图	层次方框图	流程图
核心思想	输出驱动设计，数据与程序统一描述	自顶向下分解，描述系统组成	控制流导向，描述执行步骤
主要符号	嵌套花括号 { }，逻辑符 ⊕, (n), (0,1)	矩形框、树状连线	流程框、菱形判断、箭头线
表达重点	数据的逻辑结构与对应的处理逻辑	系统的静态模块组成	操作的动态执行顺序
关系表达	集合包含、顺序、选择、循环	简单的“由...组成”	时序、分支、跳转
转换代码	非常直接（尤其到结构化代码）	不直接，需进一步设计	比较直接

需求验证

需求规格提交后，开发人员应当与用户对需求分析的结果进行**验证**，保证需求规格中定义的需求必须是**正确的**，能准确反映用户的意图。验证需求的正确性能大大减少项目后期的返工现象。

需求的正确性 需求的一致性 需求的完整性

需求的可行性 需求的必要性 需求的可验证性



需求的正确性：需求是否真实反映用户/客户意图

需求的一致性：每一项需求与其它需求不冲突矛盾

需求的完整性：验证所有可能的状态、状态变化、转入、产品和约束都无遗漏

需求的可行性：每一项需求都必须在已知系统和环境的权限和能力范围内可实施

需求的必要性：每一条需求都是用户必要的或为了符合外部系统需求所必需的

需求的可验证性：能否写出测试用例来满足需求

需求的可跟踪性：能在每一项需求与它的根源和涉及元素、源代码、测试用例之间建立起链接

最后的签字：通过评审的需求规格说明书要让用户签字，作为合同附件，对双方都具有约束力。在需求规格说明书上要求用户签字，是必须的行为，这标志着用户同意并确认该需求，意味着需求分析过程的终止以及设计阶段的开始。

PSL/PSA

PSL/PSA系统是1977年美国密执安大学开发的**问题陈述语言/问题陈述分析程序系统**，是CADSAT（计算机辅助设计和规格说明分析工具）的一部分。其中PSL是用来描述系统的形式语言，PSA是处理PSL描述的分析程序。

 **THANKS** 